AWS Deployment — Beginner Guide

Agentic Compliance Auditor

Step-by-step, cheapest-option, zero AWS experience required

What this guide gives you	Details
Total monthly cost	~$0 for first 12 months (free tier), ~$8-15/month after
AWS services used	EC2 (1 instance), Elastic IP, Security Group
Services we SKIP to save money	ALB ($16/mo), RDS, ElastiCache, OpenSearch, Bedrock, ECS
Deployment style	Single EC2 server running everything
Time to complete	About 60-90 minutes the first time
What you need before starting	Credit card, email, phone, the project code on your laptop

What's Inside

Part 1 — What AWS will cost you

Before you touch AWS, understand what you'll pay. AWS charges per hour for running servers and per GB for storage and data transfer. We're picking the smallest pieces that still work.

Free Tier (first 12 months from account creation)

- **EC2 t2.micro or t3.micro**: 750 hours/month FREE. That is enough to run one server 24/7.
- **EBS storage**: 30 GB FREE.
- **Data transfer OUT**: 100 GB/month FREE.
- **Elastic IP**: FREE while attached to a running instance.

After the free tier ends (or if you pick larger size)

Item	Monthly cost (us-east-1, approx)
t3.micro (1 GB RAM) — may OOM with embeddings	~$7.50
t3.small (2 GB RAM) — recommended minimum	~$15.00
t3.medium (4 GB RAM) — comfortable	~$30.00
30 GB EBS gp3 storage	~$2.40
Elastic IP (unattached)	~$3.60 — keep it attached to avoid this
Data transfer OUT (typical personal use)	$0-2

Note: Pick **t3.small** as your default. t3.micro is free but the embedding model loads ~400 MB and Ollama loads ~2 GB for llama3.2, which will NOT fit in 1 GB of RAM. If you must stay on free tier, use a smaller model (tinyllama or phi3:mini) — covered in Part 14.

Warning: Always set a BILLING ALARM before you do anything else. A $5 alarm will email you the moment AWS charges start — you'll set this up in Part 2.

Part 2 — Create AWS account and billing alarm

2.1 Create the account

1. Open **https://aws.amazon.com** in your browser.
2. Click the orange **Create an AWS Account** button in the top-right.
3. Enter your email, a password, and an account name (e.g., "My Compliance App").
4. Choose **Personal** account type. Fill in name, address, phone.
5. Enter a credit/debit card. AWS will charge $1 temporarily to verify.
6. Verify your phone via SMS or voice call — enter the 4-digit code they read.
7. Choose the **Basic support - Free** plan.
8. You'll get an email "Welcome to Amazon Web Services". Click the link to sign in.

2.2 Pick your region

In the top-right corner of the AWS Console, click the region name. Pick the region closest to you. Cheapest two options:

- **us-east-1 (N. Virginia)** — cheapest region, pick this if you're in the US east coast.
- **us-east-2 (Ohio)** — same price as us-east-1, pick if you prefer.
- **ap-south-1 (Mumbai)** — pick if you're in India (lower latency).

Warning: Whatever region you pick, STAY IN IT. Resources in different regions cannot talk to each other cheaply. This guide assumes us-east-1 but every step works in any region.

2.3 Set a billing alarm ($5 safety net)

1. In the top search bar, type **Billing** and click "Billing and Cost Management".
2. On the left sidebar, click **Billing preferences**.
3. Check the box **Receive Free Tier Usage Alerts**. Enter your email. Click Save.
4. In the same left sidebar, click **Budgets**.
5. Click **Create budget** → choose **Use a template** → **Zero spend budget**.
6. Enter your email address. Click **Create budget**.
7. You'll now get an email the moment your bill goes above $0.01.

Note: The Zero Spend budget is the safest option — you'll be notified immediately if anything starts costing money.

Part 3 — Create an IAM user (don't use root)

Your root account (the email you signed up with) has unlimited power and is a big target for hackers. AWS best practice is to create a normal "IAM user" and use that for everything.

1. In the AWS Console search bar, type **IAM** and click the IAM service.
2. On the left sidebar, click **Users**, then click **Create user** (top-right).
3. User name: **admin-yourname**. Check **Provide user access to the AWS Management Console**. Pick **I want to create an IAM user**. Set a password. Uncheck "User must create a new password".
4. Click **Next**. For permissions, pick **Attach policies directly**, search for **AdministratorAccess** and check it.
5. Click **Next**, then **Create user**.
6. On the success page, click **Return to users list**. Click your new user. Note the **Console sign-in URL** (looks like https://123456789012.signin.aws.amazon.com/console).
7. Sign out of the root account. Sign back in using the IAM user URL, your user name, and password.

> **Warning:** From now on, NEVER sign in with your root email. Lock the root credentials in a password manager and only use them for emergencies (like closing the account).

Part 4 — Launch your EC2 server

EC2 is the service that gives you a virtual computer in the cloud. We'll launch one small Linux server that runs everything: the API, the UI, Ollama (the LLM), and ChromaDB (the vector store).

4.1 Open the EC2 console

1. In the search bar type **EC2** and click the EC2 service.
2. On the left sidebar, click **Instances**.
3. Click the orange **Launch instances** button (top-right).

4.2 Fill the launch form (exact values to pick)

Field	What to pick
Name	compliance-auditor
AMI (Operating System)	Amazon Linux 2023 (free tier eligible). Leave default.
Architecture	64-bit (x86)
Instance type	t3.small (recommended) or t3.micro if you're on free tier
Key pair	Click Create new key pair. Name: compliance-key. Type: RSA. Forma
Network settings	Click Edit. Check all three: Allow SSH from My IP, Allow HTTP fro
Add custom rule	Click Add security group rule. Port: 8501 (Streamlit UI). Source: Anywhere. Clic
Configure storage	30 GB gp3 (free tier limit)
Advanced details	Leave everything default

Warning: Keep the **.pem key file safe**. You CANNOT download it again. If you lose it, you lose access to the server and must terminate it and start over.

1. Scroll to the bottom. Click **Launch instance**.
2. Wait 30 seconds. Click **View all instances**.
3. You should see your instance. Wait for **Instance state: Running** and **Status checks: 2/2 checks passed** (takes 1-2 minutes).

4.3 Attach an Elastic IP (so your IP doesn't change)

Without this, your server's public IP will change every time you stop/start it. Elastic IP is free while attached.

1. In the EC2 left sidebar, click **Elastic IPs**.
2. Click **Allocate Elastic IP address** → **Allocate**.
3. Select the new IP, click **Actions** → **Associate Elastic IP address**.
4. Instance: choose your compliance-auditor instance. Click **Associate**.
5. Note the IP address (like **54.123.45.67**). You'll use it everywhere.

Part 5 — Connect to your server (SSH)

5.1 On Windows (using built-in OpenSSH)

1. Open PowerShell.
2. Move your key file somewhere permanent, e.g. **C:\aws\compliance-key.pem**.
3. Run the command below to fix permissions (Windows requires strict permissions on key files).

```
icacls "C:\aws\compliance-key.pem" /inheritance:r
icacls "C:\aws\compliance-key.pem" /grant:r "$($env:USERNAME):R"
```

1. Now SSH in (replace 54.123.45.67 with YOUR Elastic IP):

```
ssh -i "C:\aws\compliance-key.pem" ec2-user@54.123.45.67
```

1. The first time, it'll ask "Are you sure you want to continue connecting?". Type **yes**.
2. You should see a prompt like **[ec2-user@ip-172-31-x-x ~]$**. You're inside the server.

5.2 On Mac/Linux

```
chmod 400 ~/Downloads/compliance-key.pem
ssh -i ~/Downloads/compliance-key.pem ec2-user@54.123.45.67
```

> **Note:** From now on, every command starting with a **$** means you run it on the server (while SSH'd in). Commands starting with **PS>** or without a prefix run on your local laptop.

Part 6 — Install Python, Ollama, dependencies

Run these commands one block at a time, on the server.

6.1 System update and basic tools

```
sudo dnf update -y
sudo dnf install -y git python3.11 python3.11-pip gcc-c++ make tar gzip
```

6.2 Install Ollama (the local LLM runner)

```
curl -fsSL https://ollama.com/install.sh | sh
sudo systemctl enable --now ollama
ollama --version
```

6.3 Pull a small model

Bigger models are smarter but use more RAM. Pick based on your instance:

Instance RAM	Recommended model	Pull command
1 GB (t3.micro)	tinyllama (~1 GB)	ollama pull tinyllama
2 GB (t3.small)	llama3.2:1b (~1.3 GB)	ollama pull llama3.2:1b
4 GB (t3.medium)	llama3.2:3b (default)	ollama pull llama3.2

For t3.small (recommended):

```
ollama pull llama3.2:1b
```

6.4 (Optional) Skip Redis — use in-memory mode

Your app already falls back to in-memory session storage when Redis isn't available. This saves money (no ElastiCache fee). No action needed.

Part 7 — Upload your project code

7.1 Option A: Use Git (easiest if your code is on GitHub)

```
cd ~
git clone https://github.com/YOUR_USERNAME/Agentic_Compliance_Auditor.git
cd Agentic_Compliance_Auditor
```

7.2 Option B: Upload from your laptop with SCP

From your LOCAL PowerShell (not the server):

```
scp -i "C:\aws\compliance-key.pem" -r "C:\All Files\Projects\AI_Projects\Agentic_Compliance_Auditor" ec2-u
```

Note: Before uploading, delete **venv/**, **data/chroma_db/**, **__pycache__** folders from your laptop copy — they're huge and will be rebuilt on the server. We'll rebuild chroma_db in Part 10.

7.3 Create a fresh virtual environment on the server

```
cd ~/Agentic_Compliance_Auditor
python3.11 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

Note: This takes 5-10 minutes. sentence-transformers and torch are big.

Part 8 — Configure for production

8.1 Create .env file on the server

```
nano ~/Agentic_Compliance_Auditor/.env
```

Paste this (adjust OLLAMA_MODEL to match what you pulled):

```
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=llama3.2:1b
EMBEDDING_MODEL=all-mpnet-base-v2

# Use in-memory fallback (no Redis needed)
REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_DB=0

API_HOST=0.0.0.0
API_PORT=8000
API_WORKERS=2

LOG_LEVEL=INFO
LOG_FILE=./logs/app.log

TOP_K_RESULTS=5
CHUNK_SIZE=256
CHUNK_OVERLAP=20
RERANK_ENABLED=false

MAX_ITERATIONS=3
AGENT_TIMEOUT=300
ENABLE_SELF_REFLECTION=false
```

Press **Ctrl+O**, Enter to save, **Ctrl+X** to exit nano.

8.2 Allow the UI to reach the API

Streamlit calls the API at http://localhost:8000 by default — that works since they're on the same box. No change needed.

8.3 Open firewall ports on the instance OS (usually already open)

Amazon Linux 2023 has no firewall daemon by default. The AWS Security Group you configured in Part 4.2 is what matters.

Part 9 — Run API and UI as background services

systemd is Linux's built-in service manager. It starts your app at boot and restarts it if it crashes.

9.1 Create the API service

```
sudo nano /etc/systemd/system/compliance-api.service
```

Paste:

```
[Unit]
Description=Compliance Auditor API
After=network.target ollama.service

[Service]
User=ec2-user
WorkingDirectory=/home/ec2-user/Agentic_Compliance_Auditor
Environment="PATH=/home/ec2-user/Agentic_Compliance_Auditor/venv/bin"
ExecStart=/home/ec2-user/Agentic_Compliance_Auditor/venv/bin/uvicorn src.api.main:app --host 0.0.0.0 --por
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

9.2 Create the UI service

```
sudo nano /etc/systemd/system/compliance-ui.service
```

Paste:

```
[Unit]
Description=Compliance Auditor UI
After=network.target compliance-api.service

[Service]
User=ec2-user
WorkingDirectory=/home/ec2-user/Agentic_Compliance_Auditor
Environment="PATH=/home/ec2-user/Agentic_Compliance_Auditor/venv/bin"
ExecStart=/home/ec2-user/Agentic_Compliance_Auditor/venv/bin/streamlit run ui/streamlit_app.py --server.po
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

9.3 Enable and start both services

```
sudo systemctl daemon-reload
sudo systemctl enable compliance-api compliance-ui
sudo systemctl start compliance-api
sudo systemctl start compliance-ui
sudo systemctl status compliance-api
sudo systemctl status compliance-ui
```

Note: Both should say **active (running)**. Press **q** to exit the status screen.

Part 10 — Ingest your compliance documents

10.1 Upload your PDFs/docs to the server

From your LOCAL PowerShell:

```
scp -i "C:\aws\compliance-key.pem" -r "C:\path\to\your\documents\*" ec2-user@54.123.45.67:~/Agentic_Compli
```

10.2 Run the ingestion script

```
cd ~/Agentic_Compliance_Auditor
source venv/bin/activate
python ingest_documents.py
```

Wait until you see a line like **"Added N chunks to vector store"**. This takes 2-5 minutes depending on document count.

10.3 Restart the API so it sees the new vector store

```
sudo systemctl restart compliance-api
```

Part 11 — Open the app to the internet

If you followed Part 4.2 exactly, ports 8000 and 8501 are already open. Test from your laptop browser:

- API docs: **http://54.123.45.67:8000/docs**
- Streamlit UI: **http://54.123.45.67:8501**

Note: Replace 54.123.45.67 with your Elastic IP from Part 4.3.

11.1 If it doesn't load

1. Check services are running: **sudo systemctl status compliance-api compliance-ui**
2. Check listening ports: **sudo ss -tlnp | grep -E '8000|8501'**
3. Check the Security Group: EC2 Console → Instances → your instance → Security tab → click the security group → Inbound rules. Ensure ports 8000 and 8501 have source **0.0.0.0/0**.
4. Check logs: **sudo journalctl -u compliance-api -n 50 --no-pager**

11.2 Lock down later (recommended)

Exposing ports 8000 and 8501 to the whole internet is fine for testing. Once it works, restrict the Security Group to your own IP (change source from 0.0.0.0/0 to My IP).

Part 12 — (Optional) Domain name + HTTPS

Only do this if you want a nice URL like https://audit.mydomain.com instead of http://54.123.45.67:8501.

12.1 Point a domain at your server

1. Buy a domain (Namecheap or Cloudflare, ~$10/year). Cheaper than AWS Route 53.
2. In your DNS provider, create an **A record**: name **audit**, value **54.123.45.67** (your Elastic IP).
3. Wait 5 minutes for DNS to propagate.

12.2 Install nginx and Let's Encrypt on the server

```
sudo dnf install -y nginx
sudo systemctl enable --now nginx
sudo dnf install -y certbot python3-certbot-nginx
```

12.3 Configure nginx to reverse-proxy to Streamlit

```
sudo nano /etc/nginx/conf.d/compliance.conf
```

```
server {
    listen 80;
    server_name audit.mydomain.com;
    location / {
        proxy_pass http://127.0.0.1:8501;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
    }
    location /api/ {
        proxy_pass http://127.0.0.1:8000/;
    }
}
```

```
sudo nginx -t
sudo systemctl reload nginx
```

12.4 Get free HTTPS certificate

```
sudo certbot --nginx -d audit.mydomain.com
```

Enter your email, agree to terms, pick redirect-to-HTTPS. Certbot auto-renews every 60 days.

> **Note:** Once HTTPS works, you can close ports 8000 and 8501 in the Security Group — only keep 80, 443, and 22 (SSH).

Part 13 — Daily operations

13.1 Useful commands

Task	Command (on the server)
Check API logs (live)	sudo journalctl -u compliance-api -f
Check UI logs (live)	sudo journalctl -u compliance-ui -f
Restart API	sudo systemctl restart compliance-api
Restart UI	sudo systemctl restart compliance-ui
Stop both	sudo systemctl stop compliance-api compliance-ui
Disk usage	df -h
RAM usage	free -h
Running processes	top (press q to quit)

13.2 Deploy new code

```
cd ~/Agentic_Compliance_Auditor
git pull              # if you used Git
source venv/bin/activate
pip install -r requirements.txt
sudo systemctl restart compliance-api compliance-ui
```

13.3 Stop the server to save money (when not in use)

1. EC2 Console → Instances → select your instance → Instance state → **Stop instance**.
2. When stopped, you pay ONLY for storage (~$2.40/month for 30GB) — no compute charge.
3. Start it again the same way when you want to use it.
4. Because you attached an Elastic IP, the IP stays the same.

Part 14 — Saving even more money

- **Stop the instance when idle.** Biggest lever. Stopping saves ~95% of the cost.
- **Use t3.micro (free tier) with tinyllama.** Smaller model, lower quality, but $0 for 12 months.
- **Use Savings Plans (after you confirm the project).** 1-year no-upfront Savings Plan saves ~30% vs on-demand.
- **Use spot instances** (advanced) — up to 70% off, but AWS can reclaim the server with 2 minutes notice. Not recommended for a persistent app.
- **Reduce embedding dimensions.** Switch back to all-MiniLM-L6-v2 (384-dim) from all-mpnet-base-v2 (768-dim) to halve vector memory.
- **Keep Elastic IP attached.** Unattached Elastic IPs cost $3.60/month each. Release ones you don't use.
- **Delete old snapshots.** If you enable backups later, stale snapshots accumulate cost.
- **Monitor with Cost Explorer.** AWS Console → Billing → Cost Explorer. Check weekly.

> **Note:** Realistic monthly cost after free tier, stopped 16 hours/day, t3.small: **~$7/month**. Always-on t3.small: **~$17/month**. Always-on t3.medium: **~$32/month**.

Part 15 — Troubleshooting

SSH says "Connection refused" or "timed out"

- Instance is still booting — wait 2 minutes.
- Security Group missing SSH rule — verify port 22, source My IP.
- Your public IP changed (home WiFi) — update the SSH rule source to your current IP.

"Permissions 0644 for key file are too open" (Windows)

Run the two **icacls** commands in Part 5.1 again.

Service fails with "ModuleNotFoundError"

- Wrong Python path in systemd unit — confirm **/home/ec2-user/Agentic_Compliance_Auditor/venv/bin/python** exists.
- **pip install -r requirements.txt** wasn't run inside the venv.

"Out of memory" / instance becomes unresponsive

- Your model is too big for the RAM. Pull a smaller one (tinyllama or llama3.2:1b).
- Resize instance: Stop → Actions → Instance settings → Change instance type → t3.medium → Start.
- Add swap: **sudo dd if=/dev/zero of=/swapfile bs=1M count=2048 && sudo chmod 600 /swapfile && sudo mkswap /swapfile && sudo swapon /swapfile**

Streamlit UI loads but says "Cannot connect to API"

- API service not running — **sudo systemctl status compliance-api**.
- Ollama not running — **sudo systemctl status ollama**.
- Wrong model name in .env — check **ollama list** vs OLLAMA_MODEL.

"Found 0 results" for every query

- Ingestion never ran or failed. Re-run **python ingest_documents.py**.
- Verify chunks exist: **ls ~/Agentic_Compliance_Auditor/data/chroma_db/** should show files.

Browser shows "ERR_CONNECTION_REFUSED"

- Security Group missing port 8501/8000 inbound rule.
- Service not running — check systemctl status.
- Streamlit bound to 127.0.0.1 only — ensure **--server.address 0.0.0.0** in unit file.

Bill showing unexpected charges

- Billing → Cost Explorer → group by Service. Find the culprit.
- Most common: unattached Elastic IP, NAT Gateway, oversized EBS, extra regions.
- Delete anything you don't recognize. Then email AWS Support — they often refund first-time accidents.

— End of guide. Good luck! —